Labsheet 7 Fullstack

SQL: DDL, DML, SELECT, Aggregation, Joins, Subqueries, Indexes, Views and Transactions

Name: Abhigyan Dubey CUID: CU24250059

Database: MariaDB 10.11 (MySQL compatible). All outputs below were produced by running the queries in order on the college_db sample data.

Database Setup

Create the database, the four tables and insert the sample data.

SQL

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE departments (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  age INT CHECK (age >= 17),
  city VARCHAR(50),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  title VARCHAR(80) NOT NULL,
  credits INT DEFAULT 3,
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE enrollments (
  student_id INT,
  course_id INT,
  marks INT,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

INSERT INTO departments VALUES (1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');

INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);

INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),
(103,203,81),
(104,201,95),(104,202,89),
(105,204,58),
(106,203,74),
(108,203,66);
```

Check: SELECT * FROM students; (8 rows returned)

Output

```
+------------+--------+------+----------+---------+
| student_id | name   | age  | city     | dept_id |
+------------+--------+------+----------+---------+
|        101 | Aarav  |   20 | Roorkee  |       1 |
|        102 | Diya   |   21 | Dehradun |       1 |
|        103 | Kabir  |   19 | Haridwar |       2 |
|        104 | Meera  |   22 | Roorkee  |       1 |
|        105 | Rohan  |   20 | Delhi    |       3 |
|        106 | Sana   |   21 | Dehradun |       2 |
|        107 | Vikram |   23 | Jaipur   |    NULL |
|        108 | Isha   |   19 | Roorkee  |       2 |
+------------+--------+------+----------+---------+
```

Part A: DDL (Data Definition Language)

A1. Create a table faculty with columns faculty_id (primary key), name (not null), email (unique), salary and dept_id (foreign key to departments).

SQL

```
CREATE TABLE faculty (
  faculty_id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  salary DECIMAL(10,2),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
DESCRIBE faculty;
```

Output

```
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| faculty_id | int(11)       | NO   | PRI | NULL    |       |
| name       | varchar(50)   | NO   |     | NULL    |       |
| email      | varchar(100)  | YES  | UNI | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
+------------+---------------+------+-----+---------+-------+
```

A2. Add a new column phone (VARCHAR(15)) to faculty using ALTER TABLE.

SQL

```
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
DESCRIBE faculty;
```

Output

```
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| faculty_id | int(11)       | NO   | PRI | NULL    |       |
| name       | varchar(50)   | NO   |     | NULL    |       |
| email      | varchar(100)  | YES  | UNI | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
| phone      | varchar(15)   | YES  |     | NULL    |       |
+------------+---------------+------+-----+---------+-------+
```

A3. Rename the column phone to contact_no.

SQL

```
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
-- (MySQL 5.7 syntax: ALTER TABLE faculty CHANGE phone contact_no VARCHAR(15);)
DESCRIBE faculty;
```

Output

```
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| faculty_id | int(11)       | NO   | PRI | NULL    |       |
| name       | varchar(50)   | NO   |     | NULL    |       |
| email      | varchar(100)  | YES  | UNI | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
| contact_no | varchar(15)   | YES  |     | NULL    |       |
+------------+---------------+------+-----+---------+-------+
```

A4. Drop the column contact_no.

SQL

```
ALTER TABLE faculty DROP COLUMN contact_no;
DESCRIBE faculty;
```

Output

```
+------------+---------------+------+-----+---------+-------+
| Field      | Type          | Null | Key | Default | Extra |
+------------+---------------+------+-----+---------+-------+
| faculty_id | int(11)       | NO   | PRI | NULL    |       |
| name       | varchar(50)   | NO   |     | NULL    |       |
| email      | varchar(100)  | YES  | UNI | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
+------------+---------------+------+-----+---------+-------+
```

A5. Remove the faculty table completely, then explain the difference between DROP, TRUNCATE and DELETE.

SQL

```
DROP TABLE faculty;
SHOW TABLES;
```

Output

```
+----------------------+
| Tables_in_college_db |
+----------------------+
| courses              |
| departments          |
| enrollments          |
| students             |
+----------------------+
```

Explanation: DROP removes the whole table, i.e. its structure, data, indexes and constraints, and it cannot be rolled back. TRUNCATE removes all rows but keeps the table structure; it is fast, resets AUTO_INCREMENT counters, cannot take a WHERE clause and is a DDL command (implicit commit). DELETE is a DML command that removes rows one by one, optionally filtered with WHERE, fires triggers and can be rolled back inside a transaction; the table structure remains.

Part B: DML (Data Manipulation Language)

B1. Insert a new student: (109, 'Neha', 20, 'Roorkee', 1).

SQL

```
INSERT INTO students VALUES (109, 'Neha', 20, 'Roorkee', 1);
SELECT * FROM students WHERE student_id = 109;
```

Output

```
+------------+------+------+---------+---------+
| student_id | name | age  | city    | dept_id |
+------------+------+------+---------+---------+
|        109 | Neha |   20 | Roorkee |       1 |
+------------+------+------+---------+---------+
```

B2. Insert a student without specifying city and observe the value stored.

SQL

```
INSERT INTO students (student_id, name, age, dept_id)
VALUES (110, 'Tara', 21, 1);
SELECT * FROM students WHERE student_id = 110;
```

Output

```
+------------+------+------+------+---------+
| student_id | name | age  | city | dept_id |
+------------+------+------+------+---------+
|        110 | Tara |   21 | NULL |       1 |
+------------+------+------+------+---------+
```

Explanation: Because city has no DEFAULT value and was not supplied, MySQL stores NULL in that column.

B3. Update the city of student 105 to 'Roorkee'.

SQL

```
UPDATE students SET city = 'Roorkee' WHERE student_id = 105;
SELECT * FROM students WHERE student_id = 105;
```

Output

```
+------------+-------+------+---------+---------+
| student_id | name  | age  | city    | dept_id |
+------------+-------+------+---------+---------+
|        105 | Rohan |   20 | Roorkee |       3 |
+------------+-------+------+---------+---------+
```

B4. Increase marks by 5 for all enrollments in course 203.

SQL

```
UPDATE enrollments SET marks = marks + 5 WHERE course_id = 203;
SELECT * FROM enrollments WHERE course_id = 203;
```

Output

```
+------------+-----------+-------+
| student_id | course_id | marks |
+------------+-----------+-------+
|        103 |       203 |    86 |
|        106 |       203 |    79 |
|        108 |       203 |    71 |
+------------+-----------+-------+
```

Explanation: Marks of course 203 were 81, 74 and 66 and are now 86, 79 and 71. The remaining exercises use this updated data.

B5. Delete the student with student_id = 109 (and any other row you added for testing).

SQL

```
DELETE FROM students WHERE student_id IN (109, 110);
SELECT COUNT(*) AS total_students FROM students;
```

Output

```
+----------------+
| total_students |
+----------------+
|              8 |
+----------------+
```

B6. Try inserting a student with age 15. Record the error message and explain which constraint caused it.

SQL

```
INSERT INTO students VALUES (111, 'Test', 15, 'Delhi', 1);
```

Output

```
--------------
INSERT INTO students VALUES (111, 'Test', 15, 'Delhi', 1)
--------------

ERROR 4025 (23000) at line 1: CONSTRAINT `students.age` failed for `college_db`.`students`
```

Explanation: The CHECK constraint on the age column (age >= 17) rejected the row because 15 is below 17. The row is not inserted and the error is reported as a CHECK constraint failure (error code 4025 in MariaDB, 3819 in MySQL 8).

Part C: Querying with SELECT

C1. Display all columns of all students.

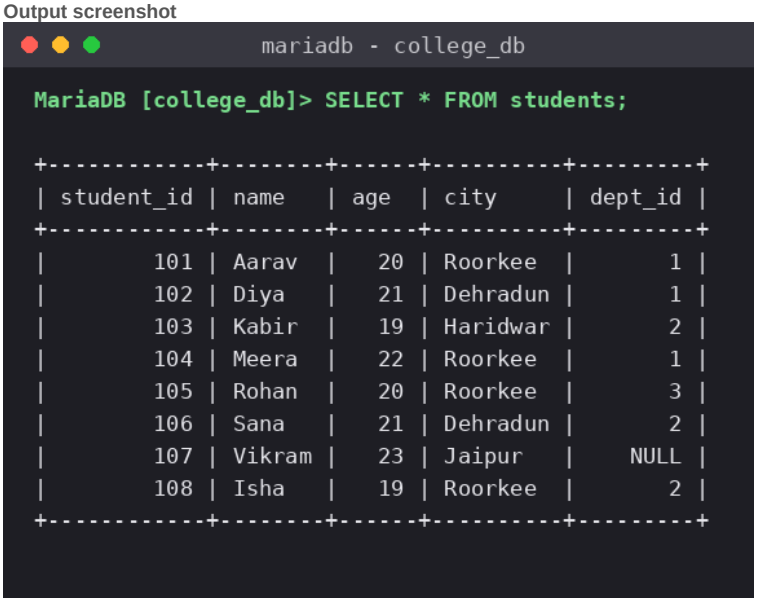
SQL

```
SELECT * FROM students;
```

Output

```
+------------+--------+------+----------+---------+
| student_id | name   | age  | city     | dept_id |
+------------+--------+------+----------+---------+
|        101 | Aarav  |   20 | Roorkee  |       1 |
|        102 | Diya   |   21 | Dehradun |       1 |
|        103 | Kabir  |   19 | Haridwar |       2 |
|        104 | Meera  |   22 | Roorkee  |       1 |
|        105 | Rohan  |   20 | Roorkee  |       3 |
|        106 | Sana   |   21 | Dehradun |       2 |
|        107 | Vikram |   23 | Jaipur   |    NULL |
|        108 | Isha   |   19 | Roorkee  |       2 |
+------------+--------+------+----------+---------+
```

Output screenshot



C2. Display only the name and city of all students.

SQL

```
SELECT name, city FROM students;
```

Output

```
+--------+----------+
| name   | city     |
+--------+----------+
| Aarav  | Roorkee  |
| Diya   | Dehradun |
| Kabir  | Haridwar |
| Meera  | Roorkee  |
| Rohan  | Roorkee  |
| Sana   | Dehradun |
| Vikram | Jaipur   |
| Isha   | Roorkee  |
+--------+----------+
```

C3. List students from Roorkee.

SQL

```
SELECT * FROM students WHERE city = 'Roorkee';
```

Output

```
+------------+-------+------+---------+---------+
| student_id | name  | age  | city    | dept_id |
+------------+-------+------+---------+---------+
|        101 | Aarav |   20 | Roorkee |       1 |
|        104 | Meera |   22 | Roorkee |       1 |
|        105 | Rohan |   20 | Roorkee |       3 |
|        108 | Isha  |   19 | Roorkee |       2 |
+------------+-------+------+---------+---------+
```

C4. List students aged between 20 and 22 (use BETWEEN).

SQL

```
SELECT * FROM students WHERE age BETWEEN 20 AND 22;
```

Output

```
+------------+-------+------+----------+---------+
| student_id | name  | age  | city     | dept_id |
+------------+-------+------+----------+---------+
|        101 | Aarav |   20 | Roorkee  |       1 |
|        102 | Diya  |   21 | Dehradun |       1 |
|        104 | Meera |   22 | Roorkee  |       1 |
|        105 | Rohan |   20 | Roorkee  |       3 |
|        106 | Sana  |   21 | Dehradun |       2 |
+------------+-------+------+----------+---------+
```

C5. List students from Roorkee or Dehradun (use IN).

SQL

```
SELECT * FROM students WHERE city IN ('Roorkee', 'Dehradun');
```

Output

```
+------------+-------+------+----------+---------+
| student_id | name  | age  | city     | dept_id |
+------------+-------+------+----------+---------+
|        101 | Aarav |   20 | Roorkee  |       1 |
|        102 | Diya  |   21 | Dehradun |       1 |
|        104 | Meera |   22 | Roorkee  |       1 |
|        105 | Rohan |   20 | Roorkee  |       3 |
|        106 | Sana  |   21 | Dehradun |       2 |
|        108 | Isha  |   19 | Roorkee  |       2 |
+------------+-------+------+----------+---------+
```

C6. List students whose name starts with 'A' (use LIKE).

SQL

```
SELECT * FROM students WHERE name LIKE 'A%';
```

Output

```
+------------+-------+------+---------+---------+
| student_id | name  | age  | city    | dept_id |
+------------+-------+------+---------+---------+
|        101 | Aarav |   20 | Roorkee |       1 |
+------------+-------+------+---------+---------+
```

C7. List students who have no department assigned (use IS NULL).

SQL

```
SELECT * FROM students WHERE dept_id IS NULL;
```

Output

```
+------------+--------+------+--------+---------+
| student_id | name   | age  | city   | dept_id |
+------------+--------+------+--------+---------+
|        107 | Vikram |   23 | Jaipur |    NULL |
+------------+--------+------+--------+---------+
```

C8. Display distinct cities of all students.

SQL

```
SELECT DISTINCT city FROM students;
```

Output

```
+----------+
| city     |
+----------+
| Roorkee  |
| Dehradun |
| Haridwar |
| Jaipur   |
+----------+
```

C9. Display students sorted by age in descending order, then by name ascending.

SQL

```
SELECT * FROM students ORDER BY age DESC, name ASC;
```

Output

```
+------------+--------+------+----------+---------+
| student_id | name   | age  | city     | dept_id |
+------------+--------+------+----------+---------+
|        107 | Vikram |   23 | Jaipur   |    NULL |
|        104 | Meera  |   22 | Roorkee  |       1 |
|        102 | Diya   |   21 | Dehradun |       1 |
|        106 | Sana   |   21 | Dehradun |       2 |
|        101 | Aarav  |   20 | Roorkee  |       1 |
|        105 | Rohan  |   20 | Roorkee  |       3 |
|        108 | Isha   |   19 | Roorkee  |       2 |
|        103 | Kabir  |   19 | Haridwar |       2 |
+------------+--------+------+----------+---------+
```

C10. Display the top 3 oldest students (LIMIT).

SQL

```
SELECT * FROM students ORDER BY age DESC LIMIT 3;
```

Output

```
+------------+--------+------+----------+---------+
| student_id | name   | age  | city     | dept_id |
+------------+--------+------+----------+---------+
|        107 | Vikram |   23 | Jaipur   |    NULL |
|        104 | Meera  |   22 | Roorkee  |       1 |
|        106 | Sana   |   21 | Dehradun |       2 |
+------------+--------+------+----------+---------+
```

C11. Display name as Student Name and age as Age using aliases.

SQL

```
SELECT name AS `Student Name`, age AS `Age` FROM students;
```

Output

```
+--------------+------+
| Student Name | Age  |
+--------------+------+
| Aarav        |   20 |
| Diya         |   21 |
| Kabir        |   19 |
| Meera        |   22 |
| Rohan        |   20 |
| Sana         |   21 |
| Vikram       |   23 |
| Isha         |   19 |
+--------------+------+
```

Part D: Functions and Aggregation

D1. Find the total number of students.

SQL

```
SELECT COUNT(*) AS total_students FROM students;
```

Output

```
+----------------+
| total_students |
+----------------+
|              8 |
+----------------+
```

D2. Find the average, minimum and maximum marks in the enrollments table.

SQL

```
SELECT AVG(marks) AS avg_marks, MIN(marks) AS min_marks, MAX(marks) AS max_marks
FROM enrollments;
```

Output

```
+-----------+-----------+-----------+
| avg_marks | min_marks | max_marks |
+-----------+-----------+-----------+
|   79.2727 |        58 |        95 |
+-----------+-----------+-----------+
```

D3. Count the number of students in each city.

SQL

```
SELECT city, COUNT(*) AS num_students FROM students GROUP BY city;
```

Output

```
+----------+--------------+
| city     | num_students |
+----------+--------------+
| Dehradun |            2 |
| Haridwar |            1 |
| Jaipur   |            1 |
| Roorkee  |            4 |
+----------+--------------+
```

D4. Find the average marks for each course.

SQL

```
SELECT course_id, AVG(marks) AS avg_marks FROM enrollments GROUP BY course_id;
```

Output

```
+-----------+-----------+
| course_id | avg_marks |
+-----------+-----------+
|       201 |   83.3333 |
|       202 |   82.5000 |
|       203 |   78.6667 |
|       204 |   58.0000 |
|       205 |   81.5000 |
+-----------+-----------+
```

D5. Show only those cities that have more than one student (HAVING).

SQL

```
SELECT city, COUNT(*) AS num_students
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Output

```
+----------+--------------+
| city     | num_students |
+----------+--------------+
| Dehradun |            2 |
| Roorkee  |            4 |
+----------+--------------+
```

D6. Find the total credits offered by each department.

SQL

```
SELECT d.dept_name, COALESCE(SUM(c.credits), 0) AS total_credits
FROM departments d
LEFT JOIN courses c ON d.dept_id = c.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output

```
+------------+---------------+
| dept_name  | total_credits |
+------------+---------------+
| CSE        |            12 |
| ECE        |             3 |
| Mechanical |             3 |
| Civil      |             0 |
+------------+---------------+
```

D7. Display student names in uppercase and the length of each name (UPPER, LENGTH).

SQL

```
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length FROM students;
```

Output

```
+------------+-------------+
| name_upper | name_length |
+------------+-------------+
| AARAV      |           5 |
| DIYA       |           4 |
| KABIR      |           5 |
| MEERA      |           5 |
| ROHAN      |           5 |
| SANA       |           4 |
| VIKRAM     |           6 |
| ISHA       |           4 |
+------------+-------------+
```

D8. Explain why WHERE cannot be used with aggregate functions, and what HAVING does differently.

SQL

```
-- Not allowed (WHERE is evaluated before grouping):
-- SELECT city, COUNT(*) FROM students WHERE COUNT(*) > 1 GROUP BY city;
-- Correct:
SELECT city, COUNT(*) AS num_students
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Output

```
+----------+--------------+
| city     | num_students |
+----------+--------------+
| Dehradun |            2 |
| Roorkee  |            4 |
+----------+--------------+
```

Explanation: WHERE filters individual rows before any grouping happens, so aggregate values such as COUNT(*) or AVG() do not exist yet at that stage. HAVING is applied after GROUP BY and filters whole groups using aggregate results. Order of execution: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY.

Part E: Joins

E1. Display each student's name along with their department name (INNER JOIN).

SQL

```
SELECT s.name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;
```

Output

```
+-------+------------+
| name  | dept_name  |
+-------+------------+
| Aarav | CSE        |
| Diya  | CSE        |
| Kabir | ECE        |
| Meera | CSE        |
| Rohan | Mechanical |
| Sana  | ECE        |
| Isha  | ECE        |
+-------+------------+
```

E2. Display all students, including those without a department (LEFT JOIN). Which student shows NULL?

SQL

```
SELECT s.name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

Output

```
+--------+------------+
| name   | dept_name  |
+--------+------------+
| Aarav  | CSE        |
| Diya   | CSE        |
| Kabir  | ECE        |
| Meera  | CSE        |
| Rohan  | Mechanical |
| Sana   | ECE        |
| Vikram | NULL       |
| Isha   | ECE        |
+--------+------------+
```

Explanation: Vikram (student 107) shows NULL because no department is assigned to him.

E3. Display all departments, including those with no students (RIGHT JOIN or LEFT JOIN with tables swapped).

SQL

```
SELECT d.dept_name, s.name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id;
```

Output

```
+------------+-------+
| dept_name  | name  |
+------------+-------+
| Civil      | NULL  |
| CSE        | Aarav |
| CSE        | Diya  |
| CSE        | Meera |
| ECE        | Kabir |
| ECE        | Sana  |
| ECE        | Isha  |
| Mechanical | Rohan |
+------------+-------+
```

Explanation: The Civil department has no students, so its student name is NULL.

E4. Display student name, course title and marks by joining students, enrollments and courses.

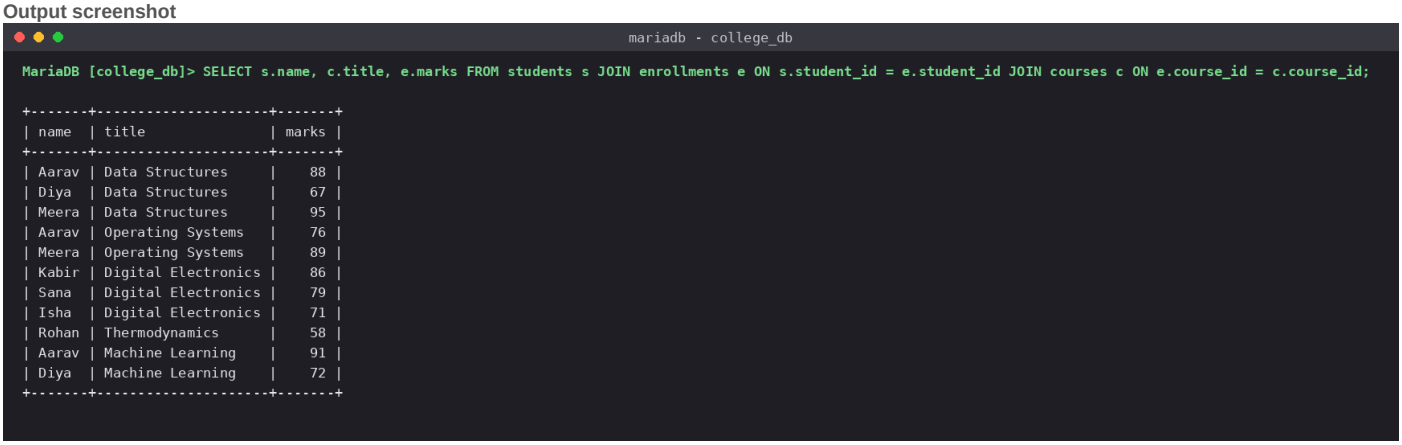
SQL

```
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

Output

```
+-------+---------------------+-------+
| name  | title               | marks |
+-------+---------------------+-------+
| Aarav | Data Structures     |    88 |
| Diya  | Data Structures     |    67 |
| Meera | Data Structures     |    95 |
| Aarav | Operating Systems   |    76 |
| Meera | Operating Systems   |    89 |
| Kabir | Digital Electronics |    86 |
| Sana  | Digital Electronics |    79 |
| Isha  | Digital Electronics |    71 |
| Rohan | Thermodynamics      |    58 |
| Aarav | Machine Learning    |    91 |
| Diya  | Machine Learning    |    72 |
+-------+---------------------+-------+
```

Output screenshot



E5. List the names of students enrolled in 'Data Structures'.

SQL

```
SELECT s.name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE c.title = 'Data Structures';
```

Output

```
+-------+
| name  |
+-------+
| Aarav |
| Diya  |
| Meera |
+-------+
```

E6. Find students who are not enrolled in any course (LEFT JOIN with IS NULL).

SQL

```
SELECT s.student_id, s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Output

```
+------------+--------+
| student_id | name   |
+------------+--------+
|        107 | Vikram |
+------------+--------+
```

E7. Display the number of students enrolled in each course (include courses with zero enrollments).

SQL

```
SELECT c.title, COUNT(e.student_id) AS num_students
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

Output

```
+--------------------+--------------+
| title              | num_students |
+--------------------+--------------+
| Data Structures    |            3 |
| Operating Systems  |            2 |
| Digital Electronics |            3 |
| Thermodynamics     |            1 |
| Machine Learning   |            2 |
+--------------------+--------------+
```

Explanation: COUNT(e.student_id) is used instead of COUNT(*) so that a course with no enrollments shows 0. In this data every course has at least one enrollment.

E8. Display each department with its number of students and its average age.

SQL

```
SELECT d.dept_name, COUNT(s.student_id) AS num_students, AVG(s.age) AS avg_age
FROM departments d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output

```
+------------+--------------+---------+
| dept_name  | num_students | avg_age |
+------------+--------------+---------+
| CSE        |            3 | 21.0000 |
| ECE        |            3 | 19.6667 |
| Mechanical |            1 | 20.0000 |
| Civil      |            0 |    NULL |
+------------+--------------+---------+
```

Part F: Subqueries

F1. Find students whose age is greater than the average age.

SQL

```
SELECT * FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

Output

```
+------------+--------+------+----------+---------+
| student_id | name   | age  | city     | dept_id |
+------------+--------+------+----------+---------+
|        102 | Diya   |   21 | Dehradun |       1 |
|        104 | Meera  |   22 | Roorkee  |       1 |
|        106 | Sana   |   21 | Dehradun |       2 |
|        107 | Vikram |   23 | Jaipur   |    NULL |
+------------+--------+------+----------+---------+
```

F2. Find the student(s) with the highest marks in any course.

SQL

```
SELECT s.name, e.course_id, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

Output

```
+-------+-----------+-------+
| name  | course_id | marks |
+-------+-----------+-------+
| Meera |       201 |    95 |
+-------+-----------+-------+
```

F3. List courses that have at least one enrollment (IN or EXISTS).

SQL

```
SELECT * FROM courses
WHERE course_id IN (SELECT course_id FROM enrollments);

-- Same result using EXISTS:
SELECT * FROM courses c
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.course_id = c.course_id);
```

Output

```
+-----------+--------------------+---------+---------+
| course_id | title              | credits | dept_id |
+-----------+--------------------+---------+---------+
|       201 | Data Structures    |       4 |       1 |
|       202 | Operating Systems  |       4 |       1 |
|       203 | Digital Electronics |       3 |       2 |
|       204 | Thermodynamics     |       3 |       3 |
|       205 | Machine Learning   |       4 |       1 |
+-----------+--------------------+---------+---------+
+-----------+--------------------+---------+---------+
| course_id | title              | credits | dept_id |
+-----------+--------------------+---------+---------+
|       201 | Data Structures    |       4 |       1 |
|       202 | Operating Systems  |       4 |       1 |
|       203 | Digital Electronics |       3 |       2 |
|       204 | Thermodynamics     |       3 |       3 |
|       205 | Machine Learning   |       4 |       1 |
+-----------+--------------------+---------+---------+
```

F4. List students enrolled in more than 2 courses.

SQL

```
SELECT name FROM students
WHERE student_id IN (
  SELECT student_id FROM enrollments
  GROUP BY student_id
  HAVING COUNT(*) > 2
);
```

Output

```
+-------+
| name  |
+-------+
| Aarav |
+-------+
```

F5. Find the department with the highest number of students.

SQL

```
SELECT d.dept_name, COUNT(*) AS num_students
FROM students s
JOIN departments d ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(*) = (
  SELECT MAX(cnt) FROM (
    SELECT COUNT(*) AS cnt FROM students
    WHERE dept_id IS NOT NULL GROUP BY dept_id
  ) t
);
```

Output

```
+-----------+--------------+
| dept_name | num_students |
+-----------+--------------+
| CSE       |            3 |
| ECE       |            3 |
+-----------+--------------+
```

F6. For each course, list students who scored above that course's average (correlated subquery).

SQL

```
SELECT c.title, s.name, e.marks
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE e.marks > (
  SELECT AVG(e2.marks) FROM enrollments e2
  WHERE e2.course_id = e.course_id
);
```

Output

```
+---------------------+-------+-------+
| title               | name  | marks |
+---------------------+-------+-------+
| Data Structures     | Aarav |    88 |
| Data Structures     | Meera |    95 |
| Operating Systems   | Meera |    89 |
| Digital Electronics | Kabir |    86 |
| Digital Electronics | Sana  |    79 |
| Machine Learning    | Aarav |    91 |
+---------------------+-------+-------+
```

Explanation: The inner query refers to e.course_id of the outer query, so it is re-evaluated for every outer row (correlated subquery).

Part G: Indexes, Views and Transactions

G1. Create an index on students(city). Run EXPLAIN SELECT * FROM students WHERE city = 'Roorkee'; and note what changes.

SQL

```
EXPLAIN SELECT * FROM students WHERE city = 'Roorkee';

CREATE INDEX idx_city ON students(city);

EXPLAIN SELECT * FROM students WHERE city = 'Roorkee';
```

Output

```
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
| id   | select_type | table    | type | possible_keys | key  | key_len | ref  | rows | Extra       |
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
|    1 | SIMPLE      | students | ALL  | NULL          | NULL | NULL    | NULL | 8    | Using where |
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
| id   | select_type | table    | type | possible_keys | key  | key_len | ref  | rows | Extra       |
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
|    1 | SIMPLE      | students | ALL  | idx_city      | NULL | NULL    | NULL | 8    | Using where |
+------+-------------+----------+------+---------------+------+---------+------+------+-------------+
```

Explanation: Before the index, the access type is ALL (full table scan) and possible_keys is NULL. After creating idx_city, possible_keys lists idx_city and the access type becomes ref when the optimizer chooses it. On a table this small the optimizer may still prefer a full scan, but on large tables the index avoids reading every row.

G2. Create a view student_marks that shows student name, course title and marks.

SQL

```
CREATE VIEW student_marks AS
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;

SELECT * FROM student_marks;
```

Output

```
+-------+---------------------+-------+
| name  | title               | marks |
+-------+---------------------+-------+
| Aarav | Data Structures     |    88 |
| Diya  | Data Structures     |    67 |
| Meera | Data Structures     |    95 |
| Aarav | Operating Systems   |    76 |
| Meera | Operating Systems   |    89 |
| Kabir | Digital Electronics |    86 |
| Sana  | Digital Electronics |    79 |
| Isha  | Digital Electronics |    71 |
| Rohan | Thermodynamics      |    58 |
| Aarav | Machine Learning    |    91 |
| Diya  | Machine Learning    |    72 |
+-------+---------------------+-------+
```

G3. Query the view to list students scoring above 80.

SQL

```
SELECT * FROM student_marks WHERE marks > 80;
```

Output

```
+-------+---------------------+-------+
| name  | title               | marks |
+-------+---------------------+-------+
| Aarav | Data Structures     |    88 |
| Meera | Data Structures     |    95 |
| Meera | Operating Systems   |    89 |
| Kabir | Digital Electronics |    86 |
| Aarav | Machine Learning    |    91 |
+-------+---------------------+-------+
```

G4. Drop the view and the index.

SQL

```
DROP VIEW student_marks;
DROP INDEX idx_city ON students;
SHOW INDEX FROM students;
```

Output

```
+----------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+
| Table    | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Ignored |
+----------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+
| students |          0 | PRIMARY  |            1 | student_id  | A         |           8 |     NULL | NULL   |      | BTREE      |         |               | NO      |
| students |          1 | dept_id  |            1 | dept_id     | A         |           8 |     NULL | NULL   | YES  | BTREE      |         |               | NO      |
+----------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+
```

Explanation: Only the PRIMARY index remains on students.

G5. Run a transaction: START TRANSACTION;, update marks for student 101 in course 201 to 100, then run ROLLBACK;. Check that the value is unchanged.

SQL

```
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
SELECT marks AS inside_transaction FROM enrollments WHERE student_id = 101 AND course_id = 201;
ROLLBACK;
SELECT marks AS after_rollback FROM enrollments WHERE student_id = 101 AND course_id = 201;
```

Output

```
+--------------------+
| inside_transaction |
+--------------------+
|                100 |
+--------------------+
+-----------------+
| after_rollback |
+-----------------+
|              88 |
+-----------------+
```

Explanation: Inside the transaction the new value 100 is visible, but ROLLBACK undoes it and the original value 88 is restored.

G6. Repeat G5 using COMMIT; instead and confirm the change is permanent.

SQL

```
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
COMMIT;
SELECT marks AS after_commit FROM enrollments WHERE student_id = 101 AND course_id = 201;
```

Output

```
+--------------+
| after_commit |
+--------------+
|          100 |
+--------------+
```

Explanation: COMMIT makes the change permanent; the value stays 100 even for new sessions.

Part H: Challenge Problems (Optional)

H1. Display a rank list of students by their total marks, with ties handled using RANK() or DENSE_RANK().

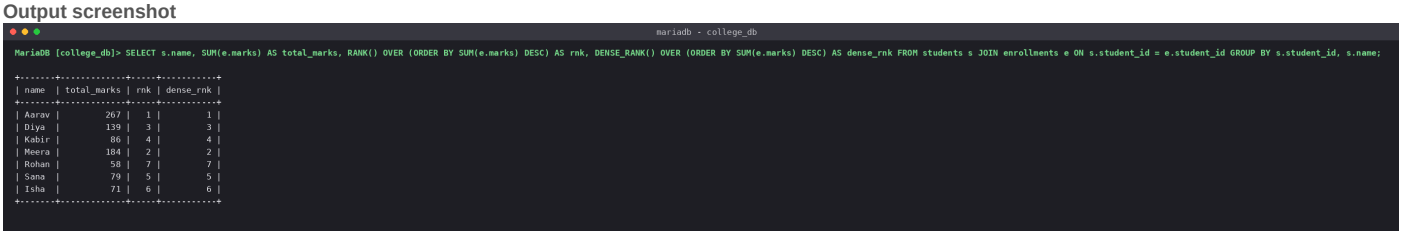
SQL

```
SELECT s.name, SUM(e.marks) AS total_marks,
       RANK() OVER (ORDER BY SUM(e.marks) DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY SUM(e.marks) DESC) AS dense_rnk
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name;
```

Output

```
+-------+-------------+-----+-----------+
| name  | total_marks | rnk | dense_rnk |
+-------+-------------+-----+-----------+
| Aarav |         267 |   1 |         1 |
| Diya  |         139 |   3 |         3 |
| Kabir |          86 |   4 |         4 |
| Meera |         184 |   2 |         2 |
| Rohan |          58 |   7 |         7 |
| Sana  |          79 |   5 |         5 |
| Isha  |          71 |   6 |         6 |
+-------+-------------+-----+-----------+
```

Output screenshot



H2. Assign a grade to each enrollment using CASE: 90 and above = A, 75 to 89 = B, 60 to 74 = C, below 60 = D.

SQL

```
SELECT student_id, course_id, marks,
  CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    WHEN marks >= 60 THEN 'C'
    ELSE 'D'
  END AS grade
FROM enrollments;
```

Output

```
+------------+-----------+-------+-------+
| student_id | course_id | marks | grade |
+------------+-----------+-------+-------+
|        101 |       201 |   100 | A     |
|        101 |       202 |    76 | B     |
|        101 |       205 |    91 | A     |
|        102 |       201 |    67 | C     |
|        102 |       205 |    72 | C     |
|        103 |       203 |    86 | B     |
|        104 |       201 |    95 | A     |
|        104 |       202 |    89 | B     |
|        105 |       204 |    58 | D     |
|        106 |       203 |    79 | B     |
|        108 |       203 |    71 | C     |
+------------+-----------+-------+-------+
```

H3. Find pairs of students from the same city (self join).

SQL

```
SELECT a.name AS student1, b.name AS student2, a.city
FROM students a
JOIN students b ON a.city = b.city AND a.student_id < b.student_id;
```

Output

```
+----------+----------+----------+
| student1 | student2 | city     |
+----------+----------+----------+
| Aarav    | Meera    | Roorkee  |
| Aarav    | Rohan    | Roorkee  |
| Aarav    | Isha     | Roorkee  |
| Diya     | Sana     | Dehradun |
| Meera    | Rohan    | Roorkee  |
| Meera    | Isha     | Roorkee  |
| Rohan    | Isha     | Roorkee  |
+----------+----------+----------+
```

Explanation: The condition a.student_id < b.student_id prevents a student from pairing with themselves and avoids duplicate mirrored pairs.

H4. Find the second highest marks in the enrollments table without using LIMIT.

SQL

```
SELECT MAX(marks) AS second_highest
FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);
```

Output

```
+----------------+
| second_highest |
+----------------+
|             95 |
+----------------+
```